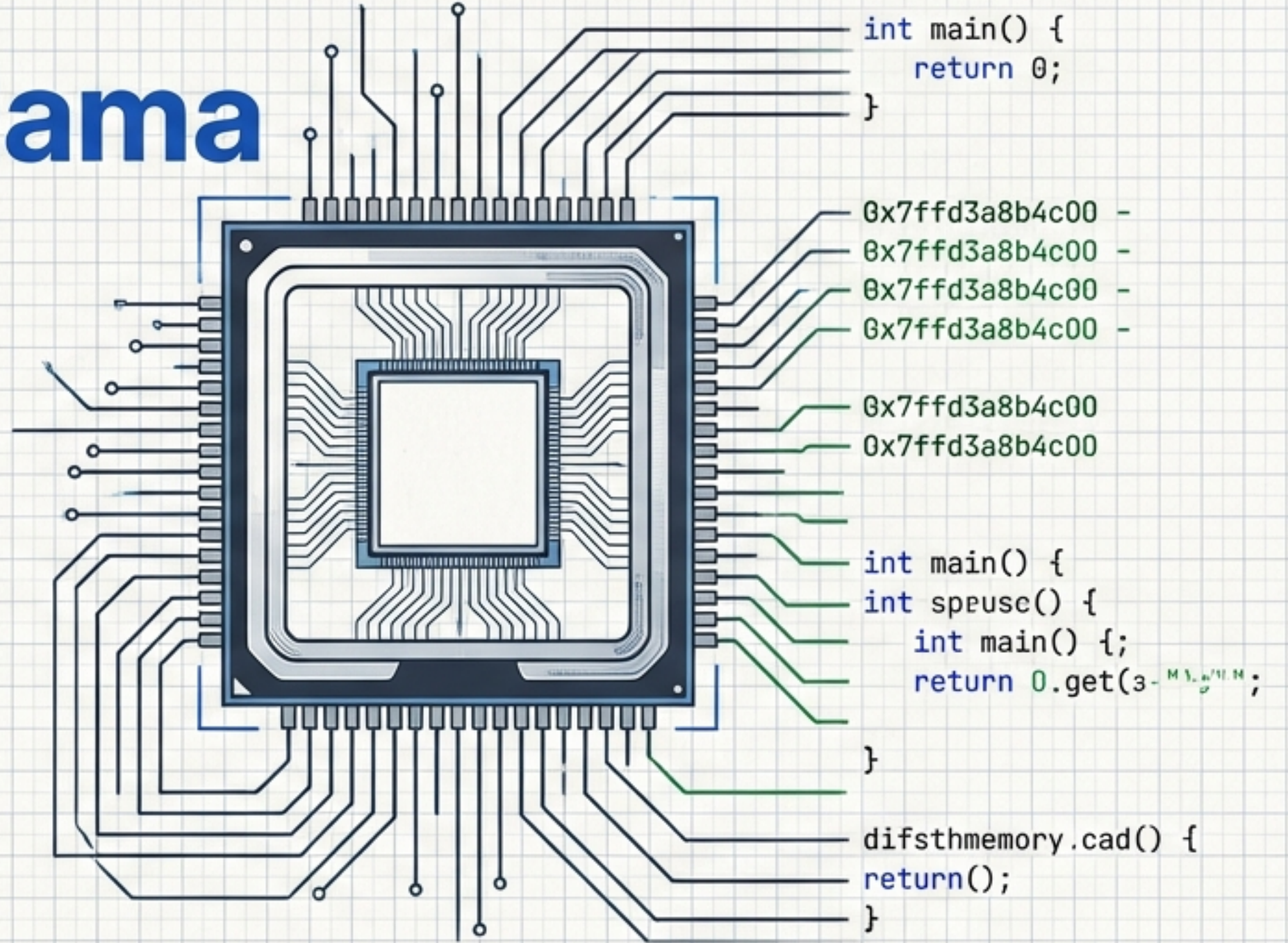


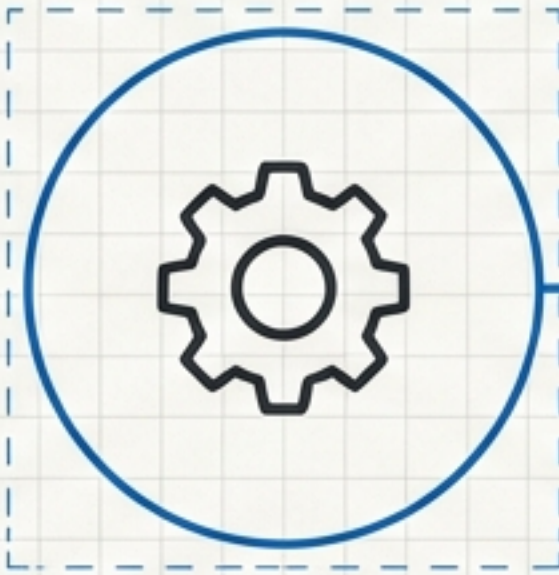
Sistem Programlama Masterclass

Bir C Programının Yaşam Döngüsü & Bellek Mimarisi

Sınav Hazırlık ve Başvuru
Rehberi (Cheat Sheet)

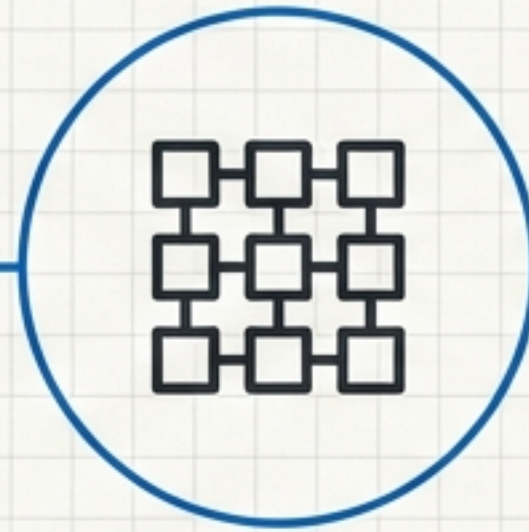


Kodlama ve İnşa



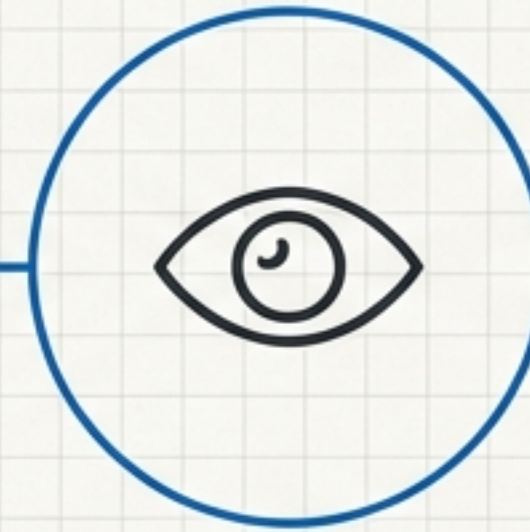
- Makine Dili ve Assembly
- C'nin Evrimi
- Derleme ve Bağlama

Mekansal Boyut



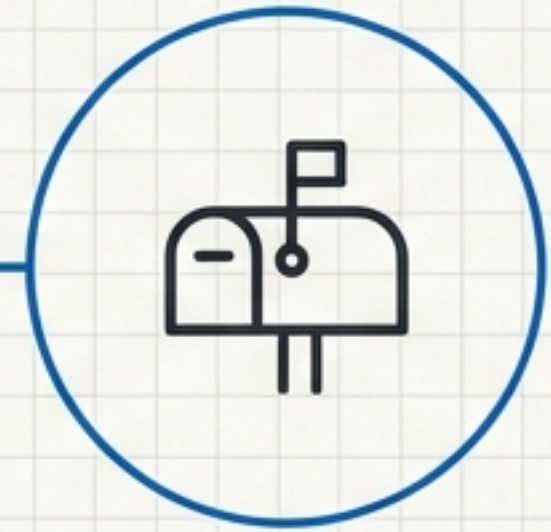
- Bellek Anatomisi
- Stack ve Heap
- Proses Yerleşimi

Aktörler ve Kurallar



- Değişken Kapsamı (Scope)
- Yaşam Süresi (Lifetime)
- Static ve Extern

Donanıma Dokunuş



- Veri Tipleri (sizeof)
- Pointer Anatomisi
- Sınav Simülasyonu

Makine Kodu (Machine Code)

```
100010001101010010101101101011
```

İşlemcinin anladığı tek dil. Sadece Opcode (işlem kodları) ve Operands (işlenenler) içerir. İnsanlar için okunması ve yazılması neredeyse imkansızdır.

Assembly Dili

```
ORG $1000  
ADD AX, BX
```

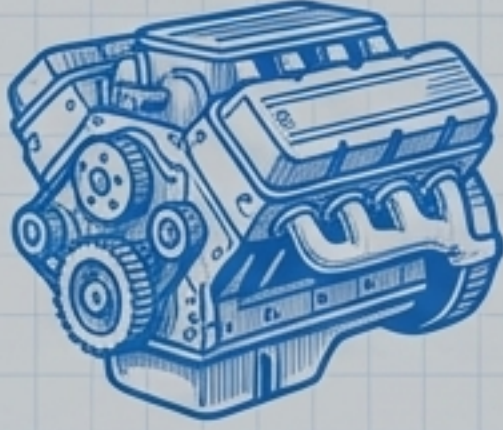
İşlemci mimarisine (örn. Intel x86 vs Motorola 68k) %100 bağımlı anımsatıcılar (mnemonics) kullanır. Fiziksel bellek adresleri sabit kodlanır. Dezavantajı: Başka donanıma taşınamaz, ekrana tek karakter basmak için 40 satır kod gerekir.

C Dili (Yüksek Seviye)

```
printf("Merhaba Dünya\n");
```

Donanımdan bağımsız, yeniden kullanılabilir mantık. C derleyicisi, fiziksel adresleri sabit kodlamak yerine, ana bellekte 'yeniden yerleştirilebilir' (relocatable) modüller oluşturur. İnsan odaklı ve modülerdir.

xxx.c Dosyaları (Gövde)



- Fonksiyonların gerçek mantığını ve gövdelerini (body) barındırır.
- Derleme sürecinde ağır işi yapar.
- Gizli tutulabilir (kapalı kaynak).
- Gizli tutulabilir (kapalı kaynak).

.h Dosyaları (Başlık / Arayüz)

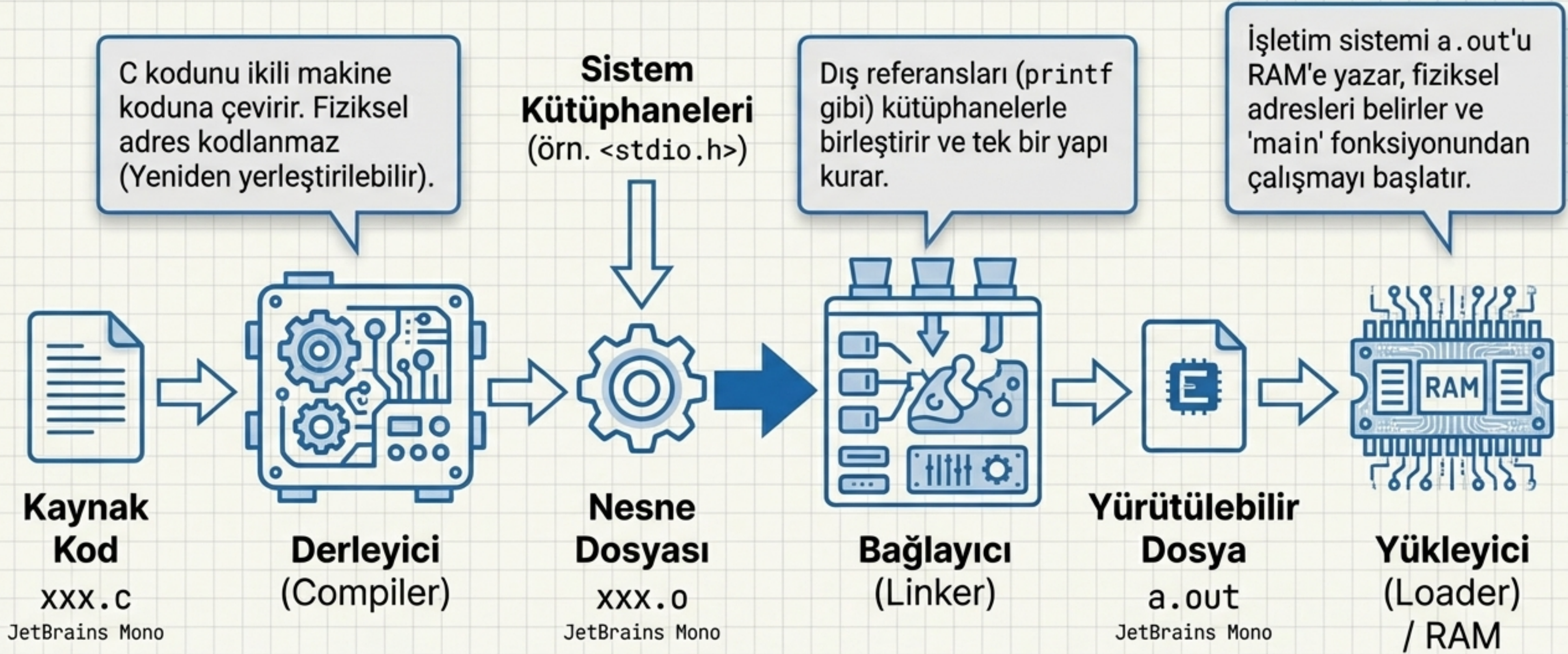


- Sadece veri türleri, fonksiyon prototipleri ve makroları içerir.
- Derleyiciye fonksiyonun doğruluğunu kanıtlar (Tip Kontrolü).
- **ASLA** fonksiyon gövdesi (işlem kodu) barındırmaz!



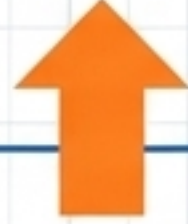
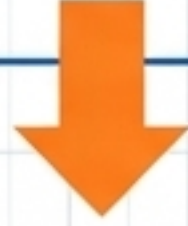
Modüler Geliştirme Kuralı: Başkasının C kaynak koduna (gövdeye) erişiminiz olmasa bile, .h dosyası sayesinde o kütüphaneyi güvenle kendi projenizde çağırabilir ve kullanabilirsiniz.

DERLEME ve BAĞLAMA SÜRECİ: KAYNAK KOD'DAN YÜRÜTÜLEBİLİR DOSYAYA



Max
(High Memory)

Stack



Heap

Data

Text

0 (Low Memory)

Yığın: Fonksiyon parametreleri, geri dönüş değerleri ve yerel değişkenler. Fonksiyon bitince silinir.

Öbek: Çalışma zamanında `malloc()` ve `free()` ile dinamik olarak ayrılan bellek alanı.

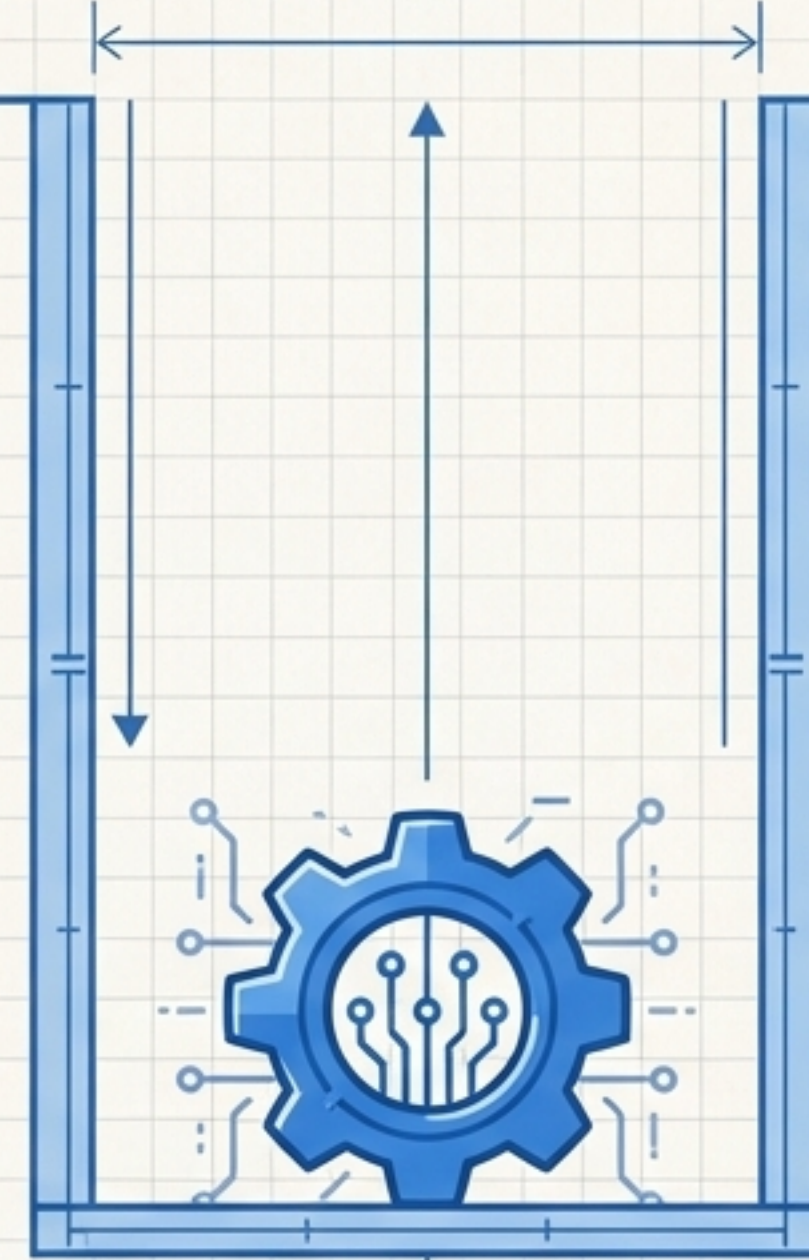
Veri Bölümü: Global ve statik değişkenleri barındırır. Program süresince yaşarlar.

Program Kodu: Makine komutları, program sayacı (PC) ve register değerlerini içerir. Sadece okunabilir.

Fonksiyon Çağrısı: Stack Yığının Anatomisi

PUSH: Fonksiyona Giriş

1. Mevcut adres kaydedilir.
2. CPU Register değerleri korunur.
3. Parametreler (Argümanlar) Stack'e eklenir.
4. Yerel değişkenler için yer ayrılır.

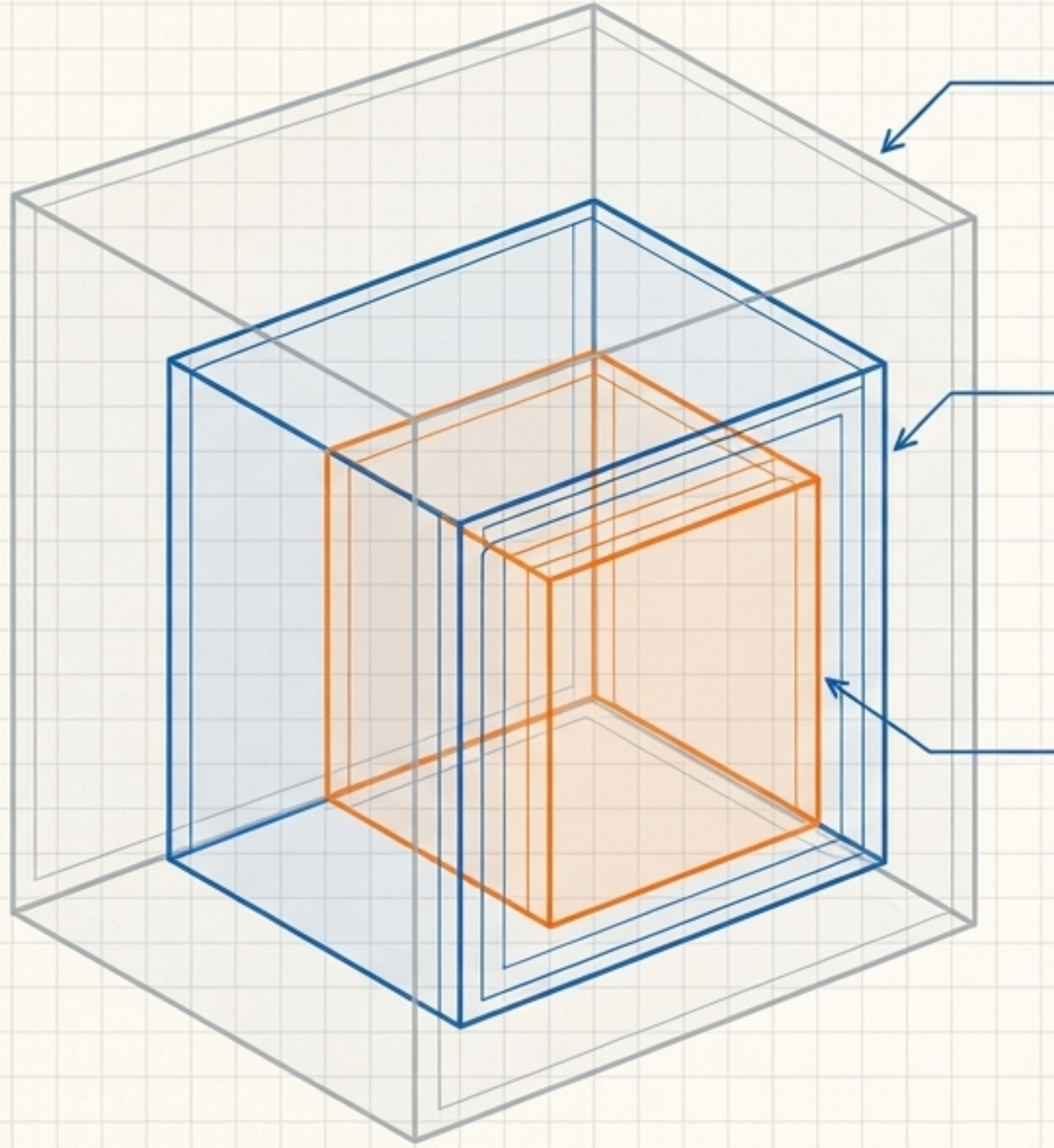


4. Kaydedilen adrese dönülerek ana programa geçilir.
3. Korunan CPU çalışma durumu geri yüklenir.
2. Parametreler (Argümanlar) kaldırılır.
1. Yerel değişkenler Stack'ten silinir.

POP: Geri Dönüş

EXECUTE: Çalışma Zamanı

- (Opsiyonel) `malloc()` ile Heap'ten dinamik bellek ayrılır.
- Fonksiyonun asıl matematiksel/mantıksal hesaplamaları yapılır.



Dosya Kapsamı (File Scope)

En dış katman. Değişken, dosyanın tamamında ve her fonksiyon tarafından görülebilir (Global).

Fonksiyon Kapsamı

Sadece tanımlandığı fonksiyonun içinde geçerlidir. Ör: `'goto start'` etiketleri.

Blok Kapsamı (Block Scope)

Yalnızca `{ }` parantezleri arasında yaşar. Döngü içi yerel değişkenler.

Altın Kural: Görünürlük Yönü

İçteki kutu, dışarıdaki kutularda tanımlı olan her şeyi görür ve kullanır. Ancak dışarıdaki kutu, içeride olup biteni asla göremez!

Kapsam (Scope) Çözümleme Tablosu

Kapsam Türü	Nerede Başlar?	Nerede Biter?	Kod Örneği
Dosya (File / Global)	Bildirim noktasında	Dosyanın sonunda	<code>int globalVar = 10;</code>
Fonksiyon (Function)	Fonksiyon gövdesi başında	Fonksiyon bitiminde	<code>start: // goto etiketi</code>
Blok (Block)	{ işaretinde	} işaretinde	<code>for(int i = 0; ...) { }</code>
Prototip (Prototype)	Parametre listesinde	Deklaratör ')' sonunda	<code>void func(int a);</code>

Sınav Notu: Prototip kapsamındaki değişken (örn. a), fonksiyonun gerçek gövdesinde geçerli değildir, yaşamı sadece o satırda biter.

Bellekte Kalma Süresi: Statik vs Otomatik

Program başladığında Data segmentine yerleşir. Program kapanana kadar asla silinmez. (Tüm global değişkenler ve fonksiyonlar).

1. Statik Süre (Global Yaşam)

Program
Başlangıcı

Program
Bitişi

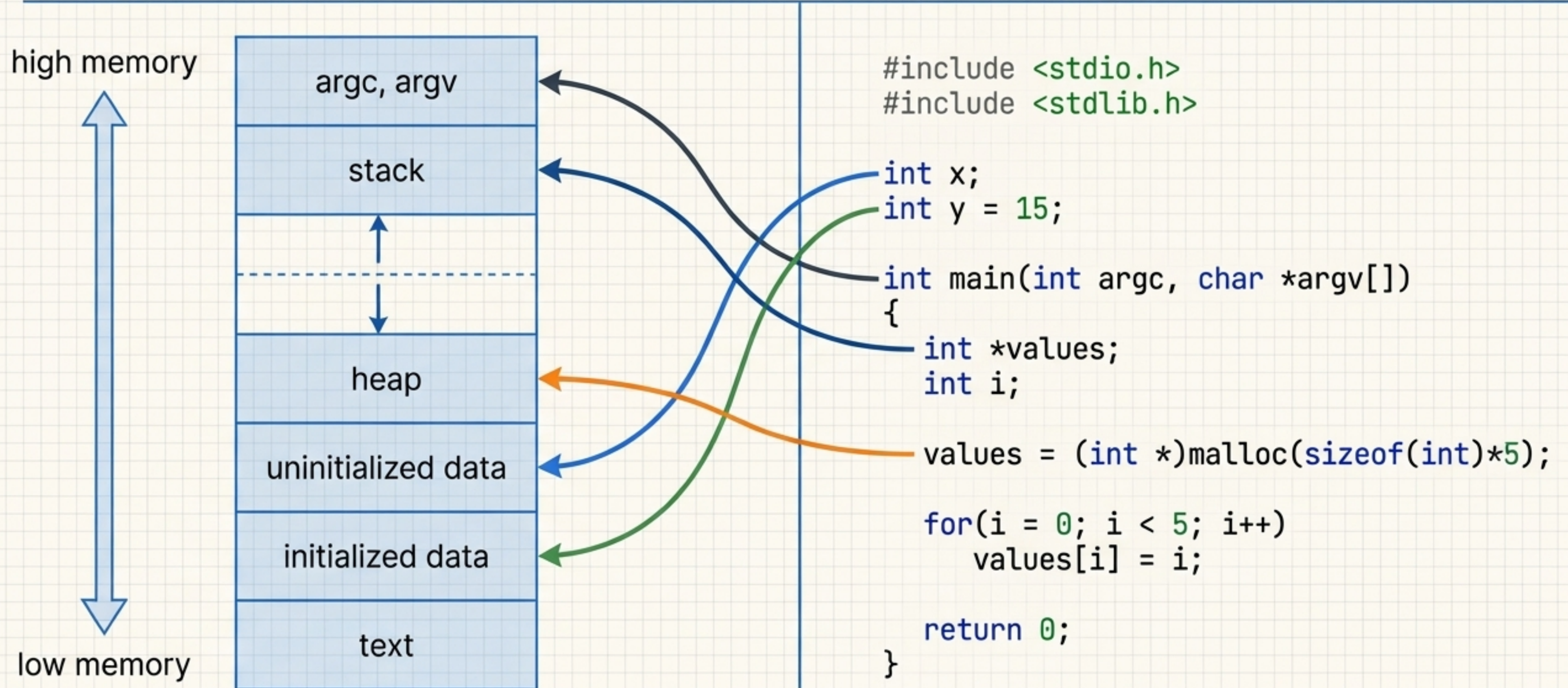
2. Otomatik Süre (Yerel Yaşam)

Fonksiyon çağrıldığında Stack'te var olur, fonksiyon bitip Pop edildiğinde kaybolur. Başlatıcısı varsa her girişte yeniden sıfırlanır.

Değişken Niteleyicileri Teşhis Matrisi

Niteleyici	Kapsam (Nereden Görülür?)	Yaşam Süresi (Ömür)	Özel Durum / Sınav Notu
Normal Yerel	Blok İçi	Otomatik (Stack)	Fonksiyon bitince değeri sıfırlanır.
Static Yerel	Blok İçi	Global (Data)	Sadece İLK çağrıda değeri atanır. Sonraki çağrılarda önceki değerini hatırlar! Varsayılan 0'dır.
Static Global	Sadece Kendi Dosyası	Global (Data)	Başka dosyalardan 'extern' ile dahi ERIŞİLEMEZ. İsim çakışmalarını izole eder.
Extern	Tüm Proje	Global (Data)	Bellekte yeni yer açmaz. "Bu değişken başka dosyada tanımlı, oraya bağlan" komutudur.

Her Şeyin Birleştiği Yer: Kodun Bellek Anatomisi



C Skaler Tipleri (Standart)

<code>char</code>	(1 byte)	Helvetica Now Display
<code>short</code>	(2 bytes)	Helvetica Now Display
<code>int</code>	(4 bytes)	Helvetica Now Display
<code>float</code>	(4 bytes)	Helvetica Now Display
<code>double</code>	(8 bytes)	Helvetica Now Display

Mimariye Bağlı Tipler (sizeof)

64-bit  32-bit

Terminal 1 (64-bit varsayılan)

```
JetBrains Mono
sizeof(long) = 8
sizeof(int *) = 8
```

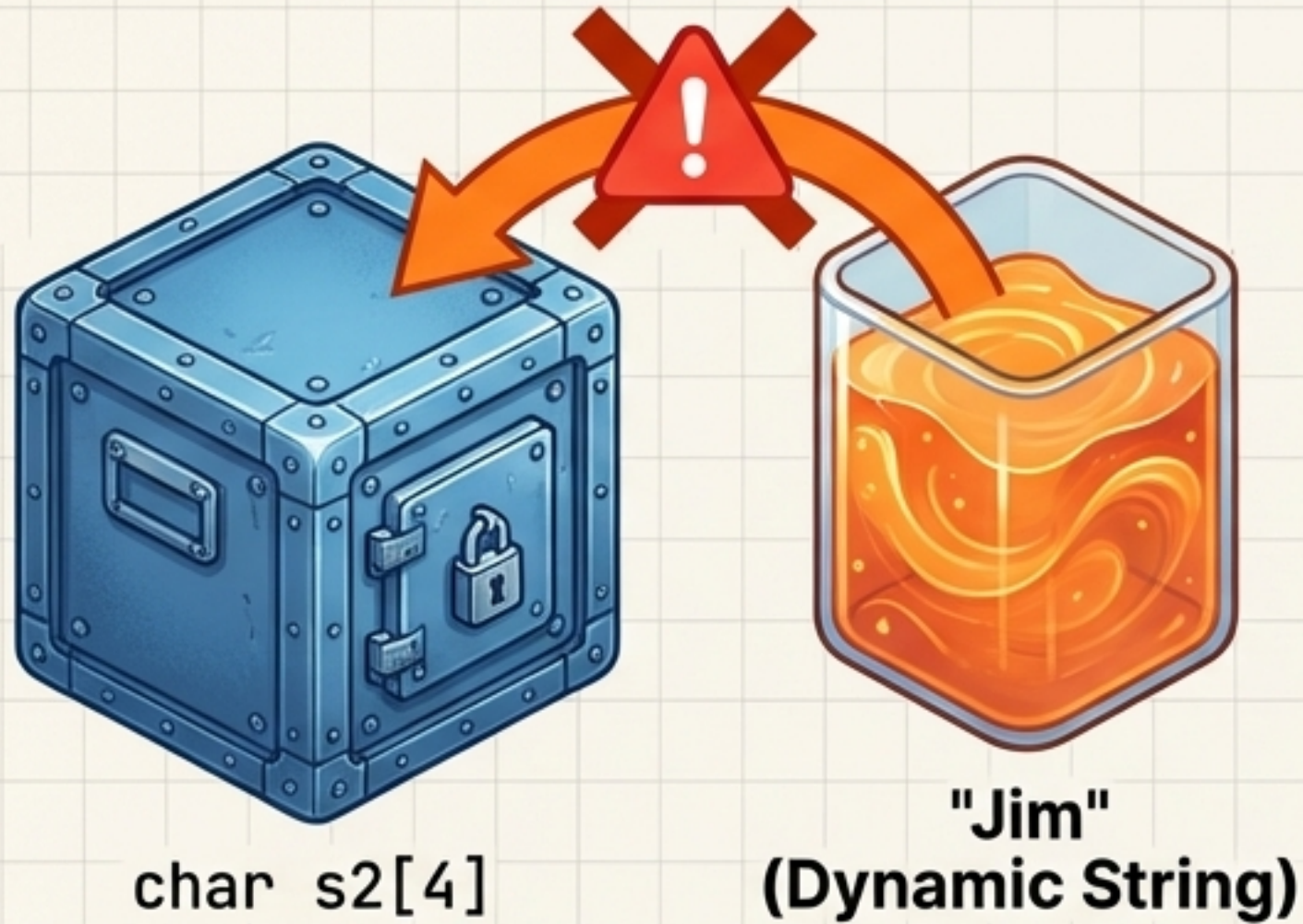
Terminal 2 (32-bit `gcc -m32`)

```
JetBrains Mono
sizeof(long) = 4
sizeof(int *) = 4
```



Kritik Bilgi: Pointer'ların (işaretçilerin) boyutu makine mimarisine sıkı sıkıya bağlıdır! Bir işaretçinin boyutu her zaman işletim sisteminin bellek adres yolunun genişliğine (32/64 bit) eşittir.

C'de Dizilerin (Aggregates) Katı Derleme Zamanı Doğası



```
char s2[4];  
s2 = "Jim"; // DERLEME HATASI
```

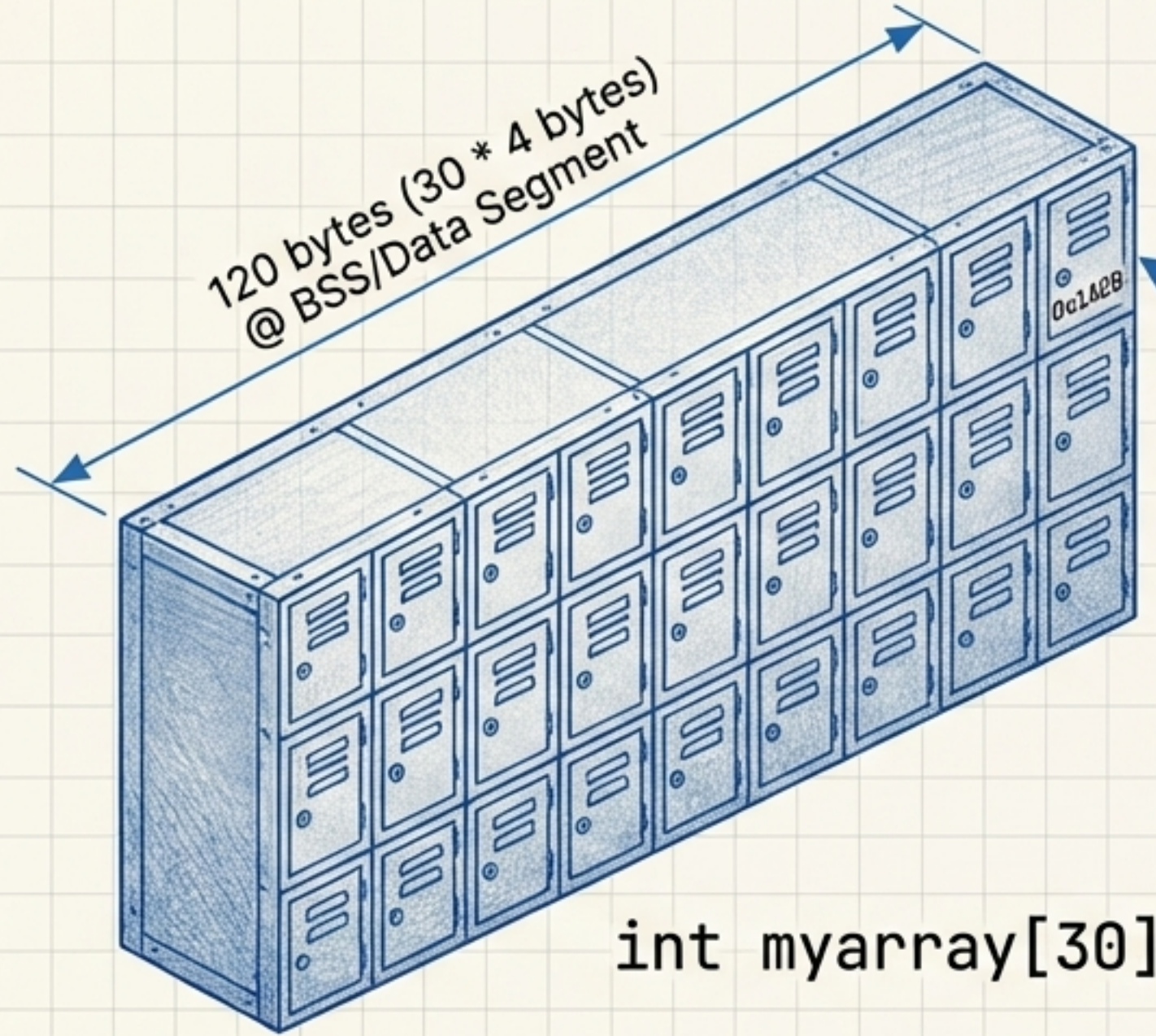
Terminal (Derleme Çıktısı)

```
error: array type 'char [4]' is not assignable
```

Neden Hata Verir?

Diziler bellekte statik olarak yer tahsis eder. s2 değişkeni aslında dizinin başını gösteren bir "sabit işaretçidir" (constant pointer). Kendi bellekteki adresini başka bir yeri gösterecek şekilde değiştiremez, sadece içindeki veriler tek tek (örn: s2[0] = 'J') değiştirilebilir.

C'de İşaretçileri (Pointers) Anlamak: Fiziksel Posta Kutusu Metaforu



Pointer aslında bir sayıdır: Sadece bellekteki başka bir adresi temsil eden bir **tam sayıdır**.



Çifte Pointer: &myarray yazıldığında, diziyi gösteren işaretçinin KENDİ posta kutusunun adresi alınır.

Dizi Mantığı: myarray[5] derleyici tarafından $*(myarray + 5)$ işlemine dönüştürülür.

Sınav Simülasyonu: Kapsam ve Ömür Sentezi

d2.c - Harici Dosya

```
char bitis = 'Z';
```

Sınır harfini belirten kaynak dosya.

d1.c - Ana Dosya

- main() fonksiyonunu barındırır.
- 'A' harfinden başlayarak, d2.c dosyasındaki 'bitis' karakterine kadar olan harfleri ekrana basmalıdır.
- Harf basma işlemi ayrı bir fonksiyon ile yapılmalıdır.
- Yazdırılan güncel harf, fonksiyonda 'static' olarak tutulmalıdır.

Soru İşaretleri (Mimari Problem):

İki ayrı dosya birbirinin değişkenini nasıl görecek?
Ve yazdırma fonksiyonu her çağrıldığında alfabede nerede kaldığını nasıl hatırlayacak?

Sınav Simülasyonu Çözümü: `extern` ve `static` Uygulaması

d2.c

```
char bitis = 'Z';
```

Global değişkendir.
Diğer dosyalara açıktır.

Bellekte yeni yer
açmaz, doğrudan
d2.c'deki adrese
bağlanır.

d1.c

```
extern char bitis;
```

```
void print_letter() {  
    static char current = 'A';  
    printf("%c", current);  
    current++;  
}
```

```
int main() { ... }
```

Static Yerel Değişken!
Belleğin Stack kısmında
değil, Data kısmında
yaşar. Fonksiyon bitip
silinse bile değerini korur
ve 'A' -> 'B' -> 'C' olarak
durumu hatırlar.